Level 2

Hackathon – Fundamentals of Web Development

Student Details

Student Name	KALAIVANI V
Register Number	24925U18013
Department	Bsc Computer science
ID	A60D8E163C458599A184879B8B01C848
College	SSS college for women
Date of Submission	28-10-2026

Git-Hub Link:

👉 **https://kalaivanikalaivani1313-sys.github.io/College-event-registration-portal-/**

1. Problem Statement

Develop a college event registration portal that enables students to explore upcoming college events, view event information, register for events, and manage their registrations through an interactive web interface. Manual registration causes crowd, data loss, and no proper tracking of event capacity.

2. Objectives

- To display all college events dynamically.
- To allow students to search and filter events by name, category, and date.
- To show detailed info of each event (title, date, venue, description).
- To provide a registration form with validation.
- To prevent duplicate registration and handle capacity limits.
- To store data in Local Storage.

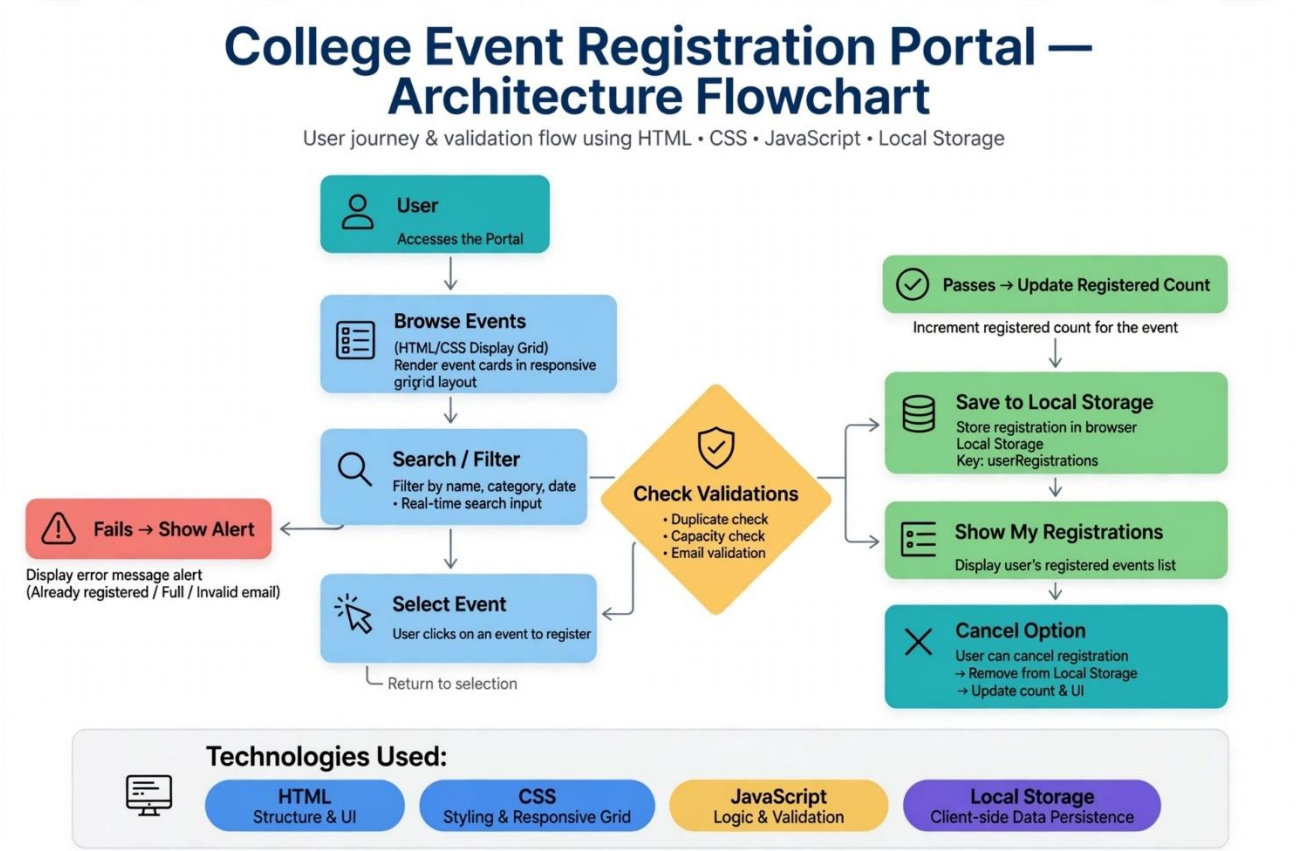
3. Architecture

- *Structure:* Single Page Application – Header, Search/Filter Section, Event Cards Grid, My Registrations Section.
- *User Navigation Flow:* `Home -> View All Events -> Search/Filter -> Click Event (View Details) -> Register Form -> Validation -> Store in Local Storage -> My

Registrations -> Cancel if needed`

- *Technologies:* HTML for structure, CSS for responsive cards, JavaScript for dynamic rendering, Local Storage for saving registration details.

- *Flow Diagram:* User -> Browse Events -> Select Event -> Check if Already Registered / Event Full? -> If No -> Register -> Update Count -> Save to Local Storage



4. Practical Implementation

We created an array of events in JavaScript `const events = [{id:1, title:"Tech Symposium", date:"2026-05-10", venue:"Main Hall", category:"Technical", desc:"…", capacity:100}, …]`. Using JS `map()` we displayed cards dynamically. Search function filters by name/category/date.

4.1 Features Implemented

- Display events dynamically with titles, dates, venues, descriptions, categories.
- Search and filter by name, category, date.
- Detailed view for each event.
- Registration form with mandatory field & email validation.
- Prevent duplicate registrations.
- Display events registered by student.
- Cancel registration where permitted.
- Local Storage for event and registration details.
- Registration count for individual events.
- Validate capacity and prevent registration when full.

4.2 Source Code

Provide the important sections of the source code and explain their functionality.

HTML

```
<input id=”search” placeholder=”Search event name”>
<select
id=”category”><option>All</option><option>Technical</option><option>Cultu
ral</option></select>
<div id=”eventList”></div>
<div id=”myEvents”></div>

<form id=”regForm”>
  <input id=”sName” required placeholder=”Student Name”>
  <input id=”sEmail” type=”email” required placeholder=”Email”>
  <button type=”submit”>Register</button>
</form>
```

CSS

```
#eventList { display:grid; grid-template-columns: repeat(3,1fr); gap:15px; }
.card { border:1px solid #ccc; padding:15px; border-radius:10px; box-
shadow:2px 2px 5px #ddd; }
.card.full { background:#ffe0e0; }
.count { font-weight:bold; color:blue; }
```

JavaScript

```
Let events = [ {id:1, title:”Code Fest”, date:”2026-05-15”, venue:”Lab 1”,
category:”Technical”, capacity:2, registered:0} ];
Let myRegs = JSON.parse(localStorage.getItem(“regs”)) || [];

Function display() {
  eventList.innerHTML = events.map(e => `
    <div class=”card ${e.registered>=e.capacity?’full’:’’}”>
      <h3>${e.title}</h3><p>${e.date} - ${e.venue}</p><p>${e.category}</p>
      <p>Registered: ${e.registered}/${e.capacity}</p>
      <button onclick=”register(${e.id})”
${e.registered>=e.capacity?’disabled’:’’}>
      ${e.registered>=e.capacity?’Full’:’Register’}</button>
    </div>`).join(“”);
}

Function register(id){
```

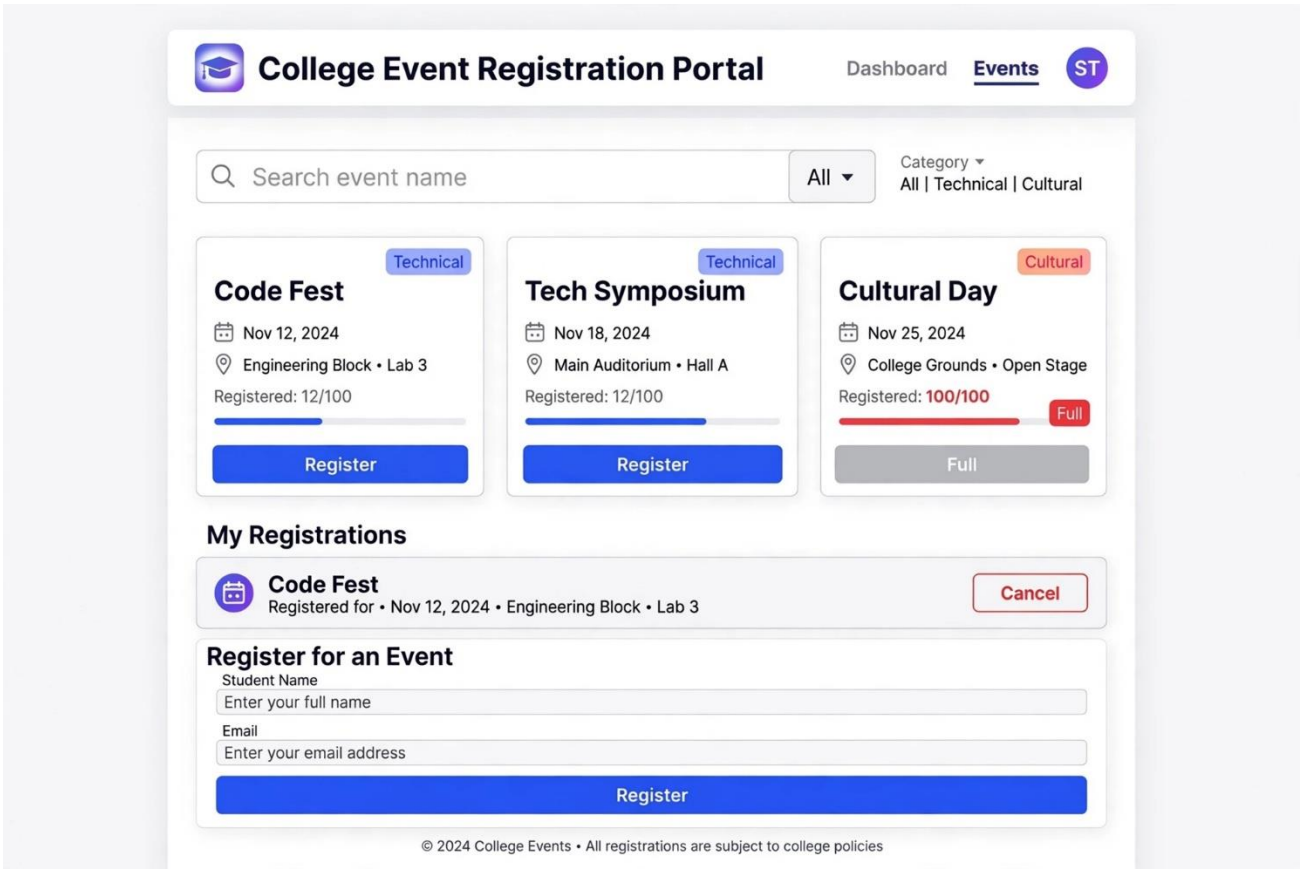
```
  If(myRegs.includes(id)){ alert("Already Registered!"); return; }
  Let ev = events.find(x=>x.id==id);
  If(ev.registered >= ev.capacity){ alert("Event Full!"); return; }
  If(sName.value==" || !sEmail.value.includes("@")){ alert("Valid details
needed"); return; }
  Ev.registered++; myRegs.push(id);
  localStorage.setItem("regs", JSON.stringify(myRegs));
  display();
}

Function cancel(id){
  Let ev = events.find(x=>x.id==id);
  Ev.registered--; myRegs = myRegs.filter(x=>x!=id);
  localStorage.setItem("regs", JSON.stringify(myRegs));
  display();
}
```

4.3 Project Execution

User opens site -> All events load from JS array -> User can search. On clicking Register, we check: 1) Empty field? 2) Valid email? 3) Already registered? 4) Capacity full? If all ok, registration count increases, ID saved in Local Storage. My Registrations section shows registered events with Cancel button.

5. Output Screenshots



College Event Portal Cards

AI & Robotics Workshop

Date: Oct 22, 2024 • 10:00 AM – 1:00 PM

Location: Innovation Lab, Block A

Registered: 42

Alumni Networking Night

Date: Oct 25, 2024 • 6:00 PM – 8:00 PM

Location: University Hall, Rooftop

Registered: 28

Event Full

Charity Hackathon

Date: Oct 30, 2024 • 9:00 AM – 5:00 PM

Location: Tech Hub, Lab 3

Registered: 60

JavaScript Event Registration — 3 Scenarios + Local Storage

Simulating user registration logic using localStorage to track registrations

Panel 1

Initial State | 0/2 Registered

0/2 Registered

Code Fest

Join the annual coding workshop • 2 spots available

Registered: 0/2 0%

Register

JavaScript Logic:

```
const max = 2;
let registrations = JSON.parse(localStorage.
getItem("codefest_registrations"); || 0;
,
```

Local Storage Saving:

```
localStorage.setItem("codefest_registrations", 0)
→ Storage updated: codefest_registrations = 0
```

Panel 2

Panel 2 — Full | 2/2 Regisstered

Alert: Event Full

Code Fest

Join the annual coding workshop • 2 spots available

Registered: 2/2 100%

Full

JavaScript Logic:

```
if (registrations >= max) {
  button.disabled = true;
  alert("Event Full");
}
```

Local Storage Saving:

```
localStorage.setItem("codefest_registrations", 2)
→ Storage updated: codefest_registrations = 2
```

Panel 3

Already Registered | Attempt Again

Alert: Already Registered

Code Fest

Join the annual coding workshop • 2 spots available

Registered: 2/2 100%

Full

JavaScript Logic:

```
if (userAlreadyRegistered) {
alert("Already Registered");
return; // prevent duplicate
}
```

Local Storage Saving:

```
localStorage.getItem("codefest_registrations") ==== 2
→ No change. Storage remains: codefest_registrations = 2
```

Key Takeaway: Use *localStorage* to persist registration count. Check if full before registering. Prevent duplicates with conditional logic.

6. Technical Explanation

- *HTML:* For layout.

- *CSS Grid:* For responsive event cards.

- *JavaScript Concepts:* Array of Objects, `map()`, `filter()`, `find()`, DOM manipulation, Local Storage (`setItem`, `getItem`, `JSON.parse/stringify`), Form Validation, Capacity logic.

- *Logic:* Duplicate check using `includes()`, Capacity check `registered >= capacity`.

7. Conclusion

We successfully implemented all expected features. The portal is fully interactive, prevents duplicate/full registrations, and uses Local Storage without needing backend. Concepts of dynamic rendering and validation learned.

8. Future Enhancements

– Add backend with database instead of Local Storage.
– Add admin panel to add/edit events.
– Send confirmation email.
– Add login for students.
– QR code for entry pass.